
Machine Learning Based Analog Circuit Fault Diagnosis
PDF 18 — Chapter 25: day13_feature_extraction.py Explained Line-by-Line
All 17 Features in Your Actual Code | Parts 1-4: Statistical through DC Level

PART 1 — Features 1-4 (Peak, RMS, Mean, Std Dev)

Chapter 25 — Complete Python Source Code Explained Part 1 — day13_feature_extraction.py (Based on your actual source code)

Before Starting

If I had to choose the single most important file in your entire project, it would be day13_feature_extraction.py. Question. Why? Because

Machine Learning

never sees the waveform. It only sees the features. Without this file, your SVM has nothing to learn from.

The Complete Pipeline

Everything before this file looked like LTspice

200 Voltage Samples

Everything after this file looks like

200 Voltage Samples

17 Physical Features

Machine Learning

This file is the bridge between

Signal Processing

and Machine Learning. Goal of the Script Professor asks What is the purpose of day13_feature_extraction.py?

Perfect Answer

This script converts each 200-point waveform into a compact numerical feature vector by extracting physically meaningful time-domain and frequency-domain characteristics. These features are then used for machine learning classification.

Import Section

Your code imports

```
import numpy as np
```

Question. Why? Because almost every feature requires mathematical operations. Examples np.max() np.mean() np.std() np.sqrt() Without NumPy, feature extraction would become much longer. Next

```
import pandas as pd
```

Why? To read the waveform dataset and save the extracted features as another CSV. Next

```
from scipy.stats import skew, kurtosis
```

Question. Why? Because two of your features are Skewness and Kurtosis. Instead of writing long mathematical equations, SciPy calculates them accurately. Next

```
from scipy.fft import  
rfft,  
rfftfreq
```

Question. Why? Because some features are extracted from the frequency domain. To do that, we first need the Fast Fourier Transform (FFT). Configuration Your code defines N_PTS = 200 Question. Why?

Because every waveform already contains 200 resampled points from Day 12. Feature extraction expects exactly 200 samples. Then T_END stores the simulation duration for each circuit. Example **RC** — 0.095 s. **Sallen-Key** — 0.020 s. Question. Why? Because frequency calculations depend on sampling interval. Different circuits have different simulation lengths, so the correct time axis must be reconstructed.

Individual Feature Functions

One thing I really like about your code is this. Instead of writing one giant function, you wrote f1() f2() f3(). f17() Each feature has its own function. This is called modular programming. Much easier to debug, modify, and explain.

Feature 1 — Peak Voltage

Your code defines

```
def f1_peak_voltage(v): return float(np.max(v))
```

Question. What is Peak Voltage? Simply the highest voltage reached by the waveform. Mathematically Peak = max(v) Example Waveform 0 1 3 5 4 2 **Peak** — 5. Very simple. Why Is It Useful? Different faults change the maximum voltage. Example An open circuit may never reach the expected peak, while a normal circuit does. Therefore

Peak Voltage

helps separate classes.

PROFESSOR ASKS

Why use Peak Voltage?

EXCELLENT ANSWER

Peak Voltage captures the maximum transient response of the circuit. Different fault conditions produce different peak amplitudes, making it a useful discriminating feature.

Feature 2 — RMS Value

Your code defines

```
def f2_rms_value(v): return float(np.sqrt(np.mean(v**2)))
```

Question. What is RMS? Root Mean Square. Formula $\sqrt{\text{mean}(v^2)}$ Question. Why square first? Because negative voltages should not cancel positive voltages. Question. Why square root at the end? To return the result to the original voltage unit. Why Is RMS Useful? Peak looks at only one point. RMS looks at the entire waveform. It represents the overall signal strength.

PROFESSOR ASKS

Why is RMS better than Peak in some situations?

EXCELLENT ANSWER

Peak considers only the maximum value, whereas RMS reflects the energy of the entire waveform. Therefore, RMS is often more robust for comparing signals with different transient shapes.

Feature 3 — Mean Voltage

Your code defines

```
def f3_mean_voltage(v): return float(np.mean(v))
```

Question. What does

Mean Voltage

measure? Simply the average voltage across all 200 samples. Formula Mean

$\sum v / N$

Question. Why useful? Some faults shift the waveform upward or downward. The mean captures that change.

Feature 4 — Standard Deviation

Your code defines

```
def f4_std_voltage(v): return float(np.std(v))
```

Question. What is Standard Deviation? It measures how much the waveform varies around its average. Example

Waveform A

5 5 5 5 **Standard deviation** — Almost zero.

Waveform B

1 9 2 8 **Standard deviation** — Large. Question. Why useful? Faults often change signal variability, not just its average.

PROFESSOR ASKS

What information does Standard Deviation provide?

EXCELLENT ANSWER

Standard Deviation measures the spread of the waveform values around the mean value. It captures signal variability, which often changes under different fault conditions.

Why These Four Features Come First

Notice something interesting. These first four features are all basic statistical descriptors. They answer four different questions.

Peak	What is the highest voltage?
RMS	What is the overall signal strength?
Mean	Where is the signal centered?
Standard Deviation	How much does the signal vary?

Together, they already describe a large part of the waveform.

Why This Design Is Good

Instead of feeding 200 raw voltage values to SVM, you summarize the waveform using physically meaningful quantities. This reduces dimensionality, improves interpretability, and makes it easier to explain why the classifier made a decision.

COMMON MISTAKES

MISTAKE: [X] Thinking Peak and RMS are the same. Peak uses one point. RMS uses all points.

MISTAKE: [X] Thinking Mean and DC Level are identical. They are not. Mean uses all 200 samples. DC Level uses only the last 20 settled samples, which is why it became so important later.

MISTAKE: [X] Thinking Standard Deviation measures noise only. It measures overall variation, which can result from transient response, oscillations, or noise.

REVISION NOTES

Remember the first four features. **Peak** — Maximum Voltage. **RMS** — Overall Signal Strength. **Mean** — Average Voltage. **Standard Deviation** — Signal Spread. These are the foundation of your feature set.

End of Chapter 25 (Part 1) You now understand the first four features in your project, their mathematics, their physical meaning, and why they help the machine learning model.

Author's Note The next continuation will explain Features 5-8: Rise Time Settling Time Skewness Kurtosis These are your first dynamic waveform features. They begin to capture how the waveform behaves over time, not just its statistical properties. They also contain several interview questions that professors commonly ask because they connect signal processing with circuit behavior.

PART 2 — Features 5-8 (Rise/Settling Time, Skew, Kurtosis)

Chapter 25 — Complete Python Source Code Explained Part 2 — Features 5 to 8 (Based on your actual source code)

Before Starting

The first four features described the waveform statistically. Now, we move to something much more interesting. These features describe how the waveform changes over time. This is where your knowledge of analog circuits starts helping the Machine Learning model.

Feature 5 — Rise Time

Your code defines

```
def f5_rise_time(t, v):
```

Question. What is Rise Time? Very simple. It measures how quickly the waveform rises from 10% to 90% of its peak voltage. Why 10% to 90%? Question. Why not 0% to 100%? Because the beginning and end of a waveform are often noisy or affected by measurement errors. Using 10% and 90% gives a much more stable measurement. This is a standard practice in electronics.

Step 1

Your code finds the peak.

```
peak = np.max(v)
```

Suppose Peak = 5V. Then **10%** — 0.5V. **90%** — 4.5V. Step 2 Your code calculates $lo = 0.1 * peak$ $hi = 0.9 * peak$ Now Python knows the two voltage levels it must search.

Step 3

Your code uses `np.argmax(v >= lo)` `np.argmax(v >= hi)` Question. Why `argmax()`? Because we need the first index where the waveform crosses 10% and 90%. Example Voltage 0 0.2 0.6 1.5 2.9 4.6 5.0
10% crosses at 0.6V. 90% crosses at 4.6V.

Rise Time

= Difference between those two times.

Special Case

Your code contains `if peak < 1e-6`: Question. Why? Imagine an open circuit. Waveform 0 0 0 0 0 There is no peak. Rise time cannot be calculated. Instead, your code returns the full simulation time. This prevents the program from crashing.

PROFESSOR ASKS

Why check for `peak < 1e-6`?

EXCELLENT ANSWER

Some fault conditions produce nearly flat waveforms. Since rise time is undefined for such signals, the code safely returns the simulation duration instead of generating an error.

Feature 6 — Settling Time

Your code defines

```
def f6_settling_time(t, v):
```

Question. What is Settling Time? It measures how long the waveform takes to settle near its final value.

Real Example

Suppose a capacitor charges. Initially 0V After some time 5V The waveform oscillates slightly before becoming stable.

Settling Time

is the time when the waveform finally stays inside a small band.

Step 1

Your code gets the final value. `final = v[-1]` Question. Why `v[-1]`? Because the last sample represents the steady-state voltage.

Step 2

Your code creates a band. `band = 0.02 * abs(final)` Question. Why 2%? Because electronics traditionally defines settling as entering and staying within $\pm 2\%$ of the final value. Example

Final Voltage

5V **Band** — $\pm 0.1V$. **Allowed range** — 4.9V. to 5.1V

Step 3

Your code uses `np.where(.)` Question. Why? To find every point outside the allowed band. Suppose the waveform becomes stable after sample 170. Everything after 170 is inside the band.

Settling Time

= Time at sample

Flat Waveform Case

Again your code checks if `abs(final) < 1e-6` Question. Why? Because if the final voltage is almost zero, the 2% band also becomes zero, making the calculation meaningless. Returning the simulation end time keeps the algorithm stable.

PROFESSOR ASKS

Why is settling time useful?

EXCELLENT ANSWER

Different faults affect how quickly a circuit reaches steady state. Settling time captures this dynamic behavior, making it an informative feature for classification.

Feature 7 — Skewness

Your code defines

```
def f7_skewness(v): return float(skew(v))
```

Question. What is Skewness? It measures whether the waveform values are symmetrical or biased toward one side. Example Perfectly symmetric distribution  **Skewness** — 0.

Positive Skew

 Long tail to the right. Positive value.

Negative Skew

 Long tail to the left. Negative value. Question. Why useful? Faults change the waveform shape, not just its amplitude. Skewness captures that asymmetry.

PROFESSOR ASKS

What does Skewness measure?

EXCELLENT ANSWER

Skewness measures the asymmetry of the waveform's value distribution. It indicates whether the waveform is biased toward higher or lower voltage values.

Feature 8 — Kurtosis

Your code defines

```
def f8_kurtosis(v): return float(kurtosis(v))
```

Question. What is Kurtosis? It measures how sharp or how flat the distribution is. Example

Sharp Distribution

■ ■ ■ ■ ■ ■ ■ ■ ■ ■ High Kurtosis.

Flat Distribution

■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ Low Kurtosis. Question. Why useful? Oscillatory waveforms and flat waveforms produce different distribution shapes. Kurtosis captures that difference.

PROFESSOR ASKS

What information does Kurtosis provide?

EXCELLENT ANSWER

Kurtosis measures the peakedness of the waveform distribution. It helps distinguish signals with sharp peaks from those with flatter distributions.

Why Features 5-8 Matter

Notice the progression. Features 1-4 describe basic statistics. Features 5-8 describe dynamic behavior and distribution shape. Together, they begin to characterize how the circuit responds, not just what values it reaches.

f5	How fast the waveform rises
f6	How long it takes to settle
f7	Distribution asymmetry
f8	Distribution peakedness

[X] Thinking Rise Time and Settling Time are the same.

Rise Time

measures how quickly the signal climbs.

Settling Time

measures how long until it becomes stable. [X] Thinking Skewness and Kurtosis describe voltage. They describe the shape of the distribution, not the voltage level itself. [X] Thinking these features are only mathematical. They have clear physical interpretations linked to circuit behavior.

REVISION NOTES

Remember the next four features. **Rise Time** — Charging Speed. **Settling Time** — Time to Become Stable. **Skewness** — Asymmetry. **Kurtosis** — Peakedness. Knowledge Check Level 1 What is Rise Time? Why use 10% and 90%? What is Settling Time? What does Skewness measure? What does Kurtosis measure? Level 2 Why is Rise Time undefined for a flat waveform? Why is a $\pm 2\%$ band used for Settling Time? How does Skewness help classify faults? Why can two waveforms have similar peaks but different kurtosis? Explain the physical meaning of Features 5–8. Professor Level Explain the implementation of Rise Time in your code. Why are edge cases handled separately in Rise Time and Settling Time? How do statistical shape descriptors complement time-domain features? Why are Features 5–8 useful for analog fault diagnosis? If you removed these features, what information would the classifier lose? End of Chapter 25 (Part 2) Congratulations. You now understand Features 5–8, including both the code implementation and the engineering concepts behind them. These features mark the transition from simple statistics to dynamic circuit behavior, making your feature set much richer and more informative.

Author's Note The next continuation will cover Features 9–13, where your project moves into the frequency domain. We'll explain: FFT (Fast Fourier Transform) Dominant Frequency Spectral Energy Spectral Centroid Bandwidth Spectral Entropy This is where your project combines digital signal processing (DSP) with machine learning, and these are among the most technically advanced features in your implementation.

PART 3 — Features 9-13 (Frequency Domain / FFT)

Chapter 25 — Complete Python Source Code Explained Part 3 — Features 9 to 13 (Frequency Domain Features) (Based on your actual source code)

Before Starting

Until now, every feature looked at the waveform in the time domain. Example **Voltage** — Time. Now we completely change our perspective. Instead of asking "How does the voltage change over time?" we ask "What frequencies exist inside this waveform?" This is called the Frequency Domain. Time Domain vs Frequency Domain Suppose someone plays a musical chord. Question. Can your ears identify different notes? Yes. Even though all notes are mixed together. Exactly the same happens with FFT. FFT separates a waveform into its frequency components. What Is FFT? FFT means Fast Fourier Transform. It converts **Voltage** — Time. into **Amplitude** — Frequency.

PROFESSOR ASKS

What is the purpose of FFT in your project?

EXCELLENT ANSWER

FFT converts the transient waveform from the time domain into the frequency domain, allowing frequency-based characteristics to be extracted as machine learning features.

Before Feature 9

Your code first computes the FFT. Conceptually it uses

```
freq = rfftfreq(...)
spec = np.abs(rfft(v))
```

Question. What happens? Input **Waveform** — FFT.

Frequency Spectrum

Example

Frequency (Hz)

0 10 20 30 40 Each frequency has its own amplitude.

Feature 9 — Dominant Frequency

Your code defines

```
def f9_dominant_frequency(...)
```

Question. What is Dominant Frequency? Simply the frequency with the highest amplitude. Example Spectrum

10 Hz

5

20 Hz

12

30 Hz

8 Largest amplitude

20 Hz

Dominant Frequency

20 Hz

Question. Why useful? Faults change the oscillation of analog circuits. Different faults produce different dominant frequencies.

PROFESSOR ASKS

Why extract Dominant Frequency?

EXCELLENT ANSWER

Different circuit faults modify the oscillatory behavior of the transient response. The dominant frequency captures the strongest frequency component and helps distinguish between fault conditions.

Feature 10 — Spectral Energy

Your code defines

```
def f10_spectral_energy(...)
```

Question. What is Spectral Energy? It measures the total energy contained in the frequency spectrum. Formula Conceptually Energy

Σ Spectrum²

Question. Why useful? Some faults produce strong oscillations. Others produce weak oscillations.

Spectral Energy

captures this difference.

PROFESSOR ASKS

How is Spectral Energy different from RMS?

EXCELLENT ANSWER

RMS measures the energy of the waveform in the time domain, whereas Spectral Energy measures how that energy is distributed across the frequency components after applying the FFT.

Feature 11 — Spectral Centroid

Your code defines

```
def f11_spectral_centroid(...)
```

Question. What is Spectral Centroid? Think of it as the center of mass of the frequency spectrum. Example Suppose most energy lies at low frequencies. **Centroid** — Low. Suppose most energy lies at high frequencies. **Centroid** — High.

Real Life Analogy

Imagine balancing a ruler. Heavy books on one side move the balance point. Exactly the same. Frequency amplitudes act like weights.

PROFESSOR ASKS

What information does Spectral Centroid provide?

EXCELLENT ANSWER

The Spectral Centroid indicates where most of the frequency energy is concentrated. It represents the average frequency weighted by spectral amplitude.

Feature 12 — Spectral Bandwidth

Your code defines

```
def f12_bandwidth(...)
```

Question. What is Bandwidth? Bandwidth measures how widely the frequency energy is spread. Example

Case 1

20 Hz

Large Peak

Everything near 20 Hz. **Bandwidth** — Small.

Case 2

5 10 20 35

50 Hz

Energy spread everywhere. **Bandwidth** — Large. Question. Why useful? Some faults create one dominant oscillation. Others create many frequency components. Bandwidth captures that difference.

PROFESSOR ASKS

Why is Bandwidth useful?

EXCELLENT ANSWER

Bandwidth measures how broadly the frequency energy is distributed. Different fault conditions produce different frequency spreads, making bandwidth an informative discriminating feature.

Feature 13 — Spectral Entropy

Your code defines

```
def f13_entropy(...)
```

Question. What is Entropy? Entropy measures disorder. Example

Case 1

One frequency contains almost all the energy. **Spectrum** — Very organized. **Entropy** — Low.

Case 2

Energy spread randomly across many frequencies. **Entropy** — High. Question. Why useful? Noise usually increases entropy. Oscillations often decrease entropy.

PROFESSOR ASKS

What does Spectral Entropy measure?

EXCELLENT ANSWER

Spectral Entropy measures the randomness or complexity of the frequency spectrum. Higher entropy indicates that energy is distributed across many frequencies rather than concentrated in a few dominant components.

Why Frequency Features Matter

Suppose two waveforms look almost identical in the time domain. Question. Can their frequency spectra still be different? Absolutely. That is why FFT features are powerful. They reveal hidden information that ordinary statistics cannot detect.

Dominant Frequency	Strongest oscillation frequency
Spectral Energy	Total energy in frequency domain
Spectral Centroid	Average frequency location
Bandwidth	Spread of frequency content
Spectral Entropy	Randomness of the spectrum

Without FFT, the classifier only knows about voltage values. With FFT, it also knows about oscillation behavior, frequency content, and spectral complexity. This provides much richer information.

COMMON MISTAKES

MISTAKE: [X] Thinking FFT changes the waveform. No. It only changes its representation.

MISTAKE: [X] Thinking Dominant Frequency means the highest frequency. Wrong. It means the frequency with the highest amplitude.

MISTAKE: [X] Thinking Spectral Energy equals RMS. They are related, but one is calculated in the time domain, the other in the frequency domain.

MISTAKE: [X] Thinking Entropy means noise only. No. Entropy measures how the spectral energy is distributed. Noise often increases it, but it is not the only factor.

REVISION NOTES

Remember the frequency features. **FFT** — Dominant Frequency. **Spectral Energy** — Spectral Centroid. **Bandwidth** — Spectral Entropy. Together, they describe the waveform from a completely different viewpoint.

KNOWLEDGE CHECK

Level 1

- What does FFT do?
- What is Dominant Frequency?
- What is Spectral Energy?
- What does Spectral Centroid measure?
- What is Spectral Entropy?

Level 2

- Why are frequency-domain features useful?
- Explain the difference between Spectral Energy and RMS.
- Why can two similar waveforms have different frequency spectra?
- How does Bandwidth help classify faults?
- Why does entropy often increase with noisy signals?

Professor Level

- Explain how FFT is used in your feature extraction pipeline.
- Why are both time-domain and frequency-domain features included?
- What physical information does each spectral feature represent?
- How would removing FFT features affect classification?
- Why is feature diversity important in machine learning?
- End of Chapter 25 (Part 3)
- Congratulations.
- You now understand Features 9-13, which introduce digital signal processing (DSP) concepts into your project.
- These features allow the classifier to learn not only how the waveform looks, but also how its frequency content changes under different fault conditions.

Author's Note The next continuation will cover Chapter 25 (Part 4): Feature 14: Crest Factor Feature 15: Peak-to-Peak Voltage Feature 16: Signal Energy Feature 17: DC Level (the feature that significantly improved your classifier) Why DC Level solved the RC_Open confusion. The complete feature vector assembly. Writing the final feature CSV. Why this became the most important improvement in your project.

PART 4 — Features 14-17 (Crest Factor through DC Level)

Chapter 25 — Complete Python Source Code Explained Part 4 — Features 14 to 17 (Final Features & Feature Vector Creation) (Based on your actual source code)

Before Starting

Congratulations. You have reached the most important part of feature extraction. Why? Because

Feature 17

changed your project. It wasn't just another feature. It solved the biggest problem your classifier had. Today we'll study the last four features.

Feature 14 — Crest Factor

Your code defines

```
def f14_crest_factor(v):
```

Question. What is Crest Factor? It measures how large the peak is compared to the overall signal. Formula $\text{Peak} \div \text{RMS}$ Example

Waveform A

Peak — 10

RMS — 5

Crest Factor

2

Waveform B

Peak — 10

RMS — 9

Crest Factor

1.11 Question. What does this mean?

Waveform A

contains a sharper peak.

Waveform B

is more uniform. Why Is Crest Factor Useful? Suppose two waveforms have the same peak. Question. Will their RMS always be the same? No. Therefore,

Crest Factor

captures information that Peak alone cannot.

PROFESSOR ASKS

Why include Crest Factor?

EXCELLENT ANSWER

Crest Factor compares the peak value with the overall signal strength. It helps distinguish waveforms containing sharp transient peaks from smoother responses.

Feature 15 — Peak-to-Peak Voltage

Your code defines

```
def f15_peak_to_peak(v):
```

Formula

Maximum Voltage

—

Minimum Voltage

Example Waveform **Maximum** — 5V. **Minimum** — -2V. **Peak-to-Peak** — 7V. Question. Why useful? Peak only tells us the highest voltage. Peak-to-Peak shows the complete voltage swing. This is especially useful for oscillatory waveforms.

PROFESSOR ASKS

How is Peak-to-Peak different from Peak?

EXCELLENT ANSWER

Peak measures only the maximum voltage, whereas Peak-to-Peak measures the total voltage range between the highest and lowest points of the waveform.

Feature 16 — Signal Energy

Your code defines

```
def f16_signal_energy(v):
```

Formula $\Sigma(v^2)$ Notice something. Earlier we studied RMS. Question. What's different? **RMS** — Average energy.

Signal Energy

Total energy. Example

Waveform A

1 1 1 1 **Energy** — 4.

Waveform B

2 2 2 2 **Energy** — 16. Question. Why useful? Faults often change the total energy stored or transferred within the circuit response.

PROFESSOR ASKS

Why include Signal Energy if RMS already exists?

EXCELLENT ANSWER

Although both are related, RMS measures the average signal magnitude, whereas Signal Energy represents the cumulative energy of the waveform. They provide complementary information.

Feature 17 — DC Level

Now we arrive at the feature that changed your project. Your updated code introduced

```
def f17_dc_level(v):  
    Why Did We Need DC Level?
```

Remember Day 16. The confusion matrix showed RC_Open being misclassified. Question. Why? Because the first 16 features did not explicitly capture the final steady-state voltage. Think about an RC circuit. At the beginning the capacitor charges. After some time, the voltage becomes almost constant. That constant value contains very important fault information. The original feature set wasn't using it directly. Implementation Your code calculates the average of the last 20 samples.

Conceptually

```
dc = np.mean(v[-20:])
```

Question. Why last 20? Because those samples represent the settled steady-state portion of the waveform. Why Not Use All 200 Samples? Good question. Suppose the waveform looks like Charging Charging Charging Stable Stable Stable If we average all 200 samples, the transient affects the result. Instead, we average only the settled region. Now DC Level really measures the final voltage.

PROFESSOR ASKS

Why average only the last 20 samples?

EXCELLENT ANSWER

The last 20 samples represent the steady-state portion of the transient response. Averaging only these samples provides a more accurate estimate of the DC level than averaging the entire waveform. Why Did DC Level Improve Accuracy? Imagine two waveforms.

Waveform A

and

Waveform B

have almost identical Peak, RMS, Energy,

Rise Time,

etc. Question. Could their final voltage still be different? Absolutely. That difference was exactly what the classifier needed. DC Level provided that missing information. Before DC Level **RC_Open** — Confused. After DC Level RC_Open

Correctly Identified

This is why

Feature 17

became the most important engineering improvement in your project.

PROFESSOR ASKS

Why was DC Level added?

EXCELLENT ANSWER

Confusion matrix analysis revealed that RC_Open faults were repeatedly misclassified. Investigation showed that steady-state voltage information was missing from the feature set. The DC Level feature was introduced to capture this information, improving fault discrimination. Creating the Feature Vector After all 17 functions finish, your code combines the outputs. Conceptually features = [f1, f2,.. f17] Question. What is this list? It is the final 17-dimensional feature vector that

Machine Learning

will receive.

One Waveform

becomes

200 Voltage Samples

17 Features

Example **Peak** — 5.2. **RMS** — 3.4. **Mean** — 2.7. **DC Level** — 4.98. Exactly 17 numbers. Processing the Entire Dataset Your code loops through every waveform in the dataset. For each sample it Extracts the waveform. Calculates all 17 features. Stores the feature vector. Copies the label. Appends the result. This repeats until every waveform has been processed. Saving the Feature Dataset Finally, your code creates a DataFrame and saves it as the feature dataset. Now instead of 200 waveform points the dataset contains 17 numerical features +

Fault Label

This is the file used by the SVM and ANN. Complete Feature Extraction Pipeline **Waveform** — **Peak**. **RMS** — **Mean**.

Standard Deviation

Rise Time

Settling Time

Skewness — Kurtosis. FFT

Dominant Frequency

Spectral Energy

Spectral Centroid

Bandwidth

Spectral Entropy

Crest Factor

Peak-to-Peak

Signal Energy

DC Level

17-Dimensional Feature Vector

This is the heart of your project. Why This File Is the Most Important Everything else depends on this file. Without it No SVM. No ANN. No GridSearchCV. No Confusion Matrix. No Dashboard. Feature extraction is the bridge between raw electrical signals and machine learning.

COMMON MISTAKES

MISTAKE: [X] Thinking DC Level equals Mean. Mean uses all samples. DC Level uses only the steady-state region.

MISTAKE: [X] Thinking Crest Factor and Peak are the same. Crest Factor depends on both Peak and RMS.

MISTAKE: [X] Thinking Signal Energy and Spectral Energy are identical. Signal Energy is calculated in the time domain. Spectral Energy is calculated after FFT in the frequency domain.

REVISION NOTES

Remember the final four features. **Crest Factor** — Sharpness of Peaks. **Peak-to-Peak** — Voltage Swing. **Signal Energy** — Total Time-Domain Energy. **DC Level** — Steady-State Voltage. Memory Trick Remember this sentence. "Peak shape, voltage swing, total energy, final voltage." These four ideas summarize Features 14-17.

KNOWLEDGE CHECK

Level 1

- What is Crest Factor?
- How is Peak-to-Peak calculated?
- Why is Signal Energy different from RMS?
- What does DC Level measure?
- Why are only the last 20 samples used?

Level 2

- Explain why DC Level improved RC_Open classification.
- Why is Crest Factor useful even when Peak is already available?
- Compare Signal Energy and Spectral Energy.
- Why is the feature vector more useful than the raw waveform?
- Describe the complete feature extraction pipeline.

Professor Level

- Explain all 17 features and justify why each was selected.
- Why did the addition of DC Level improve the classifier?
- How does feature engineering influence machine learning performance?
- Which feature do you consider the most important, and why?
- If you had to add an 18th feature, what would you choose and why?
- End of Chapter 25
- Congratulations.
- You have now mastered the entire feature extraction module.
- You understand:
 - Every one of the 17 features.
 - Their mathematical definitions.
 - Their physical interpretations.
 - Their implementation in code.
 - Their contribution to machine learning.
- Why DC Level became the turning point in your project.
- This chapter is the core of your project because it transforms raw electrical measurements into meaningful engineering information that the classifier can learn from.

Author's Note The next continuation begins Chapter 26 — `day14_train_svm.py`. We'll explain: Reading the feature dataset. Train-test split. Label encoding. StandardScaler. Why SVM needs scaling. Creating the baseline SVM. Training the model. Making predictions. Evaluating accuracy. Understanding every important line of your SVM training code. This chapter marks the transition from feature engineering to machine learning training.